

From: <https://chatgpt.com/c/6a8a20f0-f3fc-83ee-891b-ba811975393f>

you asked: go through

message time: 2026-08-23 03:49:11

go through the previous chats from start to end completely and responses how detailed-
<https://chatgpt.com/c/6a7f33eb-07f4-83ee-8229-383f952dcf93> and <https://chatgpt.com/c/6a8a1a24-1218-83e9-9f55-db7ec639b9a5> .

And continue further according to previous -

For your goal, I recommend turning this into a structured “**2-Year Java + Spring Boot Product Company Interview Bible**” with:

Module 1: 150 Core Java questions + answers

Module 2: 120 Spring/Spring Boot questions + internal working

Module 3: 80 JPA/Hibernate questions

Module 4: 75 REST/API questions

Module 5: 75 Microservices questions

Module 6: 50 Spring Security/JWT questions

Module 7: 50 SQL questions + queries

Module 8: 50 Multithreading questions

Module 9: 50 Kafka + Redis questions

Module 10: 50 Docker/AWS questions

Module 11: 100 DSA problems specifically for 2-year Java SDE interviews

Module 12: 50 LLD/system-design scenarios

Module 13: 100 project/resume follow-up questions

Module 14: Company-specific patterns for **Amazon, Google, Microsoft, Adobe, Uber, Walmart, Mastercard, Morgan Stanley, Cisco, Salesforce, Atlassian, Oracle, etc.**

Most importantly, for every question I'd structure it as:

Question → Short interview answer → Detailed explanation → Real project example → Follow-up question → Tricky interviewer counter-question → Best answer → Code/diagram where relevant

That format will train you to **answer follow-up questions instead of just memorizing definitions.**

Module 3 — JPA / Hibernate

80 Questions + Answers

The important shift here will be from:

Spring Boot

to:

Spring Data JPA
↓
JPA
↓
Hibernate
↓
Persistence Context
↓
Entity Lifecycle
↓
Dirty Checking
↓
First-Level Cache
↓
Second-Level Cache
↓
Lazy/Eager Loading
↓
N+1
↓
JPQL / Criteria / Native Query
↓
Fetch Join
↓
Optimistic/Pessimistic Locking
↓
Transactions + JPA

And for the JPA module, the particularly high-value questions will be **Persistence Context, Entity Lifecycle, dirty checking, save() internals, flush() vs commit(), findById() vs getReferenceById(), LAZY vs EAGER, N+1, JOIN FETCH, cascading, orphan removal, optimistic locking, and why Hibernate generates unexpected SQL**.**

chatgpt response: I'll use t

I'll use the two conversation links as the primary source, reconcile them with the continuation you pasted, and then continue Module 3 in the same depth and interview-oriented format.

I checked both conversation links directly. They currently open only the ChatGPT login/share shell, not the actual conversation messages, so I cannot truthfully claim that I read those two chats from beginning to end. ([ChatGPT](#))

I also checked the available conversation/library files for an exported copy of those chats, but none was available.

I **do** have the continuation context from our previous work, including the structure and the point where we reached Module 3. So I'll continue from exactly that point rather than restarting or pretending I saw content that I could not access.

Agreed master structure

Module 1: 150 Core Java questions + answers **Module 2:** 120 Spring/Spring Boot questions + internal working **Module 3:** 80 JPA/Hibernate questions **Module 4:** 75 REST/API questions **Module 5:** 75 Microservices questions **Module 6:** 50 Spring Security/JWT questions **Module 7:** 50 SQL questions + queries **Module 8:** 50 Multithreading questions **Module 9:** 50 Kafka + Redis questions **Module 10:** 50 Docker/AWS questions **Module 11:** 100 DSA problems for 2-year Java SDE interviews **Module 12:** 50 LLD/System Design scenarios **Module 13:** 100 project/resume follow-up questions **Module 14:** Company-specific patterns — Amazon, Google, Microsoft, Adobe, Uber, Walmart, Mastercard, Morgan Stanley, Cisco, Salesforce, Atlassian, Oracle, etc.

And the intended format remains:

Question → Short interview answer → Detailed explanation → Real project example → Follow-up question → Tricky interviewer counter-question → Best answer → Code/diagram where relevant

For Module 3, the core chain we established is:

```
text
Spring Data JPA
    ↓
    JPA
    ↓
    Hibernate
    ↓
Persistence Context
```

```
↓
Entity Lifecycle
↓
Dirty Checking
↓
First-Level Cache
↓
Second-Level Cache
↓
Lazy / Eager Loading
↓
N+1
↓
JPQL / Criteria / Native SQL
↓
Fetch Join
↓
Optimistic / Pessimistic Locking
↓
Transactions
```

Module 3 — JPA / Hibernate

Q1. What is JPA?

Short interview answer

JPA (Jakarta Persistence API) is a specification that defines how Java objects are mapped to relational database tables and how persistence operations are performed. It is not an implementation. Hibernate is the most commonly used JPA implementation.

Detailed explanation

JPA gives us standard APIs and annotations such as:

```
java
@Entity
@Table
@Id
@GeneratedValue
@OneToMany
```

```
@ManyToOne
@OneToOne
@ManyToMany
```

and interfaces such as:

```
java
EntityManager
EntityTransaction
TypedQuery
```

JPA defines **what should happen**.

Hibernate, EclipseLink, etc. define **how it actually happens**.

Think of it as:

```
text
JPA
└─ Specification / Contract

Hibernate
└─ Implementation of that contract
```

For example:

```
java
@Entity
public class Employee {

    @Id
    @GeneratedValue
    private Long id;

    private String name;
}
```

The `@Entity` annotation is defined by JPA.

Hibernate reads it and performs the actual ORM work.

Real project example

Suppose your application contains:

text

Employee Java object

↓

Hibernate

↓

employee table

Instead of manually writing:

sql

```
INSERT INTO employee(name) VALUES (?);
```

you can do:

java

```
employeeRepository.save(employee);
```

and Hibernate generates the SQL required to persist the entity.

Follow-up question

Is Hibernate the same thing as JPA?

No.

text

JPA = specification

Hibernate = implementation

Tricky interviewer counter-question

Can I use JPA without Hibernate?

Yes.

JPA is an API/specification. Another provider such as EclipseLink can implement it.

Best interview answer

JPA is a standard persistence specification provided by Jakarta. Hibernate is an implementation of that specification. Spring Data JPA sits on top of JPA and simplifies repository-based data access.

Q2. What is Hibernate?

Short interview answer

Hibernate is an ORM framework and a JPA implementation that maps Java objects to relational database tables and handles SQL generation, persistence context, dirty checking, caching, fetching and transaction-related persistence operations.

Detailed explanation

Without ORM, you typically do:

```
java
Connection
PreparedStatement
ResultSet
```

and manually convert database rows into Java objects.

Hibernate abstracts much of this.

```
text
Java Object
    ↓
Hibernate ORM
    ↓
SQL
    ↓
Database
```

For example:

```
java
Employee employee =
    entityManager.find(Employee.class, 101L);
```

Hibernate may execute:

```
sql
SELECT
    id,
    name,
    salary
FROM employee
WHERE id = ?
```

and construct/populate the entity.

Important Hibernate responsibilities

Hibernate handles things such as:

```
text
Object ↔ Relational Mapping
SQL generation
Entity lifecycle
Persistence Context
Dirty checking
First-level cache
Lazy loading
Association management
Batching
Fetching
Second-level cache integration
```

Follow-up

Why use Hibernate instead of JDBC?

JDBC gives lower-level database interaction.

Hibernate gives ORM and persistence management.

However, Hibernate does not eliminate SQL knowledge.

A strong Java developer should understand:

```
text
Hibernate
+
SQL
+
Database execution
```

because poor ORM usage can produce inefficient SQL.

Q3. What is ORM?

Short interview answer

ORM stands for Object-Relational Mapping. It maps Java objects to relational database tables and translates object operations into SQL/database operations.

Example

Java class:

```
java
public class Employee {

    private Long id;
    private String name;
    private Double salary;
}
```

Database:

```
text
employee
-----
id          BIGINT
name        VARCHAR
salary      DECIMAL
```

Mapping:

```
text
Employee.id      ↔ employee.id
Employee.name    ↔ employee.name
Employee.salary  ↔ employee.salary
```

Hibernate performs this mapping.

Real project example

Instead of:

```
sql
SELECT * FROM employee WHERE id = 10;
```

then manually:

```
java
Employee e = new Employee();
e.setId(resultSet.getLong("id"));
e.setName(resultSet.getString("name"));
```

you can use:

```
java
employeeRepository.findById(10L);
```

Tricky question

Does ORM mean you never write SQL?

No.

You may still write:

```
text
JPQL
Native SQL
Criteria API
Specifications
custom queries
```

And SQL knowledge remains extremely important for performance debugging.

Q4. What is Spring Data JPA?

Short interview answer

Spring Data JPA is a Spring abstraction built on top of JPA that reduces boilerplate repository code and provides features such as derived queries, pagination, sorting and repository implementations generated automatically.

Hierarchy:

```
text
Spring Data JPA
    ↓
    JPA
    ↓
    Hibernate
    ↓
    JDBC
    ↓
    Database Driver
    ↓
    Database
```

Example:

```

java
public interface EmployeeRepository
    extends JpaRepository<Employee, Long> {

    List<Employee> findByDepartment(String department);

}

```

You do not implement:

```

java
findByDepartment()

```

Spring Data derives the query from the method name.

Conceptually it may produce something similar to:

```

sql
SELECT *
FROM employee
WHERE department = ?

```

Real project example

```

java
employeeRepository.findByDepartment("IT");

```

instead of writing repository boilerplate manually.

Tricky interviewer question

Does Spring Data JPA replace Hibernate?

No.

Usually:

```

text
Spring Data JPA
    ↓
JPA API
    ↓
Hibernate implementation

```

Q5. Explain the relationship between JPA, Hibernate and Spring Data JPA.

Best interview answer

This is one of the most important questions in a Spring Boot interview.

```
text
Spring Data JPA
    ↓
simplifies
    ↓
JPA
    ↓
specification/API
    ↓
Hibernate
    ↓
implementation
    ↓
JDBC
    ↓
Database
```

More precisely:

JPA

Defines the persistence standard.

Examples:

```
java
@Entity
@Id
@OneToMany
EntityManager
```

Hibernate

Provides the actual ORM engine.

Responsibilities include:

```
text
mapping
SQL generation
```

```
dirty checking
persistence context
lazy loading
caching
```

Spring Data JPA

Provides repository abstractions:

```
java
JpaRepository
CrudRepository
PagingAndSortingRepository
```

So:

```
text
JPA != Hibernate
Hibernate != Spring Data JPA
```

but they commonly work together.

Q6. What is an Entity?

Short interview answer

An entity is a Java class whose instances are managed by the JPA persistence provider and are mapped to database records.

Example:

```
java
@Entity
@Table(name = "employee")
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
```

```
private Double salary;
}
```

Hibernate understands:

```
text
Employee object
    ↓
employee row
```

Important requirement

An entity normally needs an identifier:

```
java
@Id
private Long id;
```

The identifier uniquely identifies the entity within persistence.

Q7. What happens internally when you call `repository.save(entity)`?

This is a **very high-value 2-year experience interview question**.

Short interview answer

`save()` eventually delegates to JPA's `EntityManager.persist()` or `EntityManager.merge()` depending on whether Spring Data JPA considers the entity new.

Conceptually:

```
text
repository.save(entity)
    ↓
SimpleJpaRepository.save()
    ↓
isNew(entity)
    ↓
persist() OR merge()
    ↓
EntityManager
    ↓
Hibernate Session
    ↓
```

Persistence Context

↓

SQL at flush

Important distinction

For a new entity:

```
java
entityManager.persist(entity);
```

For an existing/detached entity:

```
java
entityManager.merge(entity);
```

Example

```
java
Employee employee = new Employee();
employee.setName("Rahul");

employeeRepository.save(employee);
```

For a new entity, Spring Data typically invokes:

```
java
entityManager.persist(employee);
```

Now the entity becomes managed.

But this does **not necessarily mean INSERT executes immediately**.

That leads directly to one of the most important concepts:

```
text
persist()
  ↓
Persistence Context
  ↓
managed entity
  ↓
flush()
```

↓
SQL INSERT

Tricky interviewer question

Does `save()` immediately execute INSERT SQL?

Not necessarily.

The entity can become managed first, while SQL execution occurs during a later flush.

For example:

```
java
employeeRepository.save(employee);

employee.setSalary(100000.0);
```

Hibernate may detect the final state during flush and generate the corresponding SQL.

Q8. What is the Persistence Context?

Short interview answer

The persistence context is a managed set of entity instances maintained by the JPA `EntityManager`. It acts as the first-level cache and guarantees identity within that context: for a given entity type and ID, the same managed Java object instance is reused.

This is a critical concept.

Suppose:

```
java
Employee e1 =
    entityManager.find(Employee.class, 10L);

Employee e2 =
    entityManager.find(Employee.class, 10L);
```

Within the same persistence context:

```
java
e1 == e2
```

will normally be:


```
text
true
```

because Hibernate already has the entity managed.

Conceptually:

```
text
Persistence Context

Employee#10
  ↓
Java object
```

Second lookup:

```
text
find(Employee#10)
  ↓
already managed?
  ↓
YES
  ↓
return same object
```

rather than executing another SQL query.

Q9. What is the First-Level Cache in Hibernate?

Short interview answer

The first-level cache is the cache associated with the Hibernate Session/JPA Persistence Context. It is enabled by default and cannot normally be disabled.

Example:

```
java
Employee e1 =
    entityManager.find(Employee.class, 1L);

Employee e2 =
    entityManager.find(Employee.class, 1L);
```

The first call may execute:

```
sql
SELECT ...
FROM employee
WHERE id = 1;
```

The second call can retrieve the already-managed entity from the persistence context.

```
text
First find()
    ↓
DB
    ↓
Persistence Context

Second find()
    ↓
Persistence Context
```

Important distinction

First-level cache is:

```
text
per Persistence Context / Session
```

not:

```
text
global for entire application
```

So another transaction/session normally has another persistence context.

Q10. Is the First-Level Cache the same as Persistence Context?

Best interview answer

They are closely related but conceptually not identical.

The **Persistence Context** is the JPA concept representing the set of managed entities.

The **first-level cache** is the Hibernate terminology describing the caching behavior inside the Session.

A good interview explanation is:

In JPA terminology I would say Persistence Context; in Hibernate terminology the Session's first-level cache provides that identity/caching behavior.

Q11. What are the entity lifecycle states in JPA?

There are four states you should know extremely well:

```
text
TRANSIENT
    ↓ persist()
MANAGED
    ↓ detach()/clear()/close()
DETACHED
    ↓ remove()
REMOVED
```

1. Transient

Object exists only in Java memory.

```
java
Employee e = new Employee();
e.setName("A");
```

It is not associated with a persistence context.

```
text
Java memory
    ↓
not managed
```

2. Managed

```
java
entityManager.persist(e);
```

Now:

```
text
e
    ↓
Persistence Context
```

Hibernate tracks it.

3. Detached

```
java
entityManager.detach(e);
```

or:

```
java
entityManager.clear();
```

or the persistence context closes.

The Java object still exists, but Hibernate is no longer tracking it.

4. Removed

```
java
entityManager.remove(e);
```

The entity is scheduled for deletion.

Q12. What is Dirty Checking?

Short interview answer

Dirty checking is Hibernate's mechanism for detecting changes made to managed entities and generating SQL UPDATE statements during flush without requiring an explicit update method.

This is one of the most frequently asked Hibernate internal questions.

Example:

```
java
@Transactional
public void updateEmployee(Long id) {

    Employee e =
        entityManager.find(Employee.class, id);

    e.setSalary(120000.0);
}
```

Notice:

```
java
repository.save(e);
```

is not required for the update.

Why?

Because:

```
text
find()
↓
managed entity
↓
modify entity
↓
dirty checking
↓
flush()
↓
UPDATE
```

Hibernate keeps track of the entity's state.

Conceptually:

```
text
Original state:
salary = 100000

Current state:
salary = 120000

Difference detected
↓
UPDATE employee
SET salary = 120000
WHERE id = ?
```

Very important

Dirty checking generally works for **managed entities**.

If the entity is detached:

```
java
entityManager.detach(e);
e.setSalary(120000.0);
```

Hibernate is no longer automatically tracking the modification.

Q13. Does Hibernate compare every field of every object all the time?

Interview-oriented answer

Not in the sense of continuously running a database comparison.

Hibernate tracks managed entities within the persistence context and, during flush, determines whether their state has changed.

Historically Hibernate commonly maintains a snapshot of entity state and compares managed state against that snapshot during dirty checking.

Modern Hibernate has optimizations, but the key interview concept remains:

```
text
managed entity
    ↓
change in memory
    ↓
flush
    ↓
dirty checking
    ↓
SQL UPDATE if needed
```

Q14. What is `flush()`?

Short interview answer

`flush()` **synchronizes the in-memory state of the persistence context with the database by executing the SQL statements required to synchronize changes.**

Example:

```
java
entityManager.persist(employee);
entityManager.flush();
```

At flush, Hibernate may issue:

```
sql
INSERT INTO employee ...
```

But remember

`flush()` is **not the same as commit**.

```
text
flush()
  ↓
send/execute SQL
  ↓
database state synchronized
```

while:

```
text
commit()
  ↓
transaction successfully finalized
```

A flushed transaction can still be rolled back.

Q15. What is the difference between `flush()` and `commit()`?

This is another extremely common interview question.

<code>flush()</code>	<code>commit()</code>
Synchronizes Persistence Context with DB	Finalizes transaction
Executes pending SQL	Commits transaction
Does not necessarily end transaction	Ends successful transaction
Transaction may still be rolled back	Successful commit makes changes durable according to DB transaction semantics
Managed entities remain in persistence context	Transaction lifecycle completes

Example:

```

java

employee.setSalary(100000);

entityManager.flush();

// SQL has been synchronized
// transaction is still active

throw new RuntimeException();

```

If the transaction rolls back:

```

text

UPDATE
    ↓
ROLLBACK

```

So:

```

text

flush ≠ commit

```

Memorize that distinction.

Q16. When does Hibernate automatically flush?

Hibernate can flush depending on the configured flush mode and circumstances. In the common default behavior, it may flush before transaction commit and before executing certain queries where synchronization is required.

The simplified interview model is:

```

text

persist/update/delete
    ↓
Persistence Context
    ↓
flush triggered
    ↓
SQL

```

Common situations include:

text

```
transaction commit  
certain query executions  
explicit entityManager.flush()
```

Tricky question

Does every call to `save()` cause immediate SQL?

No.

`save()` and SQL execution are separate concepts.

Q17. What is `EntityManager`?

Short interview answer

`EntityManager` is the JPA interface used to interact with the persistence context and perform entity operations such as persist, find, merge, remove, detach and flush.

Example:

java

```
@PersistenceContext  
private EntityManager entityManager;
```

Operations:

java

```
entityManager.persist(entity);  
  
entityManager.find(Employee.class, id);  
  
entityManager.merge(entity);  
  
entityManager.remove(entity);  
  
entityManager.flush();  
  
entityManager.clear();
```

Q18. Difference between `persist()` and `merge()`?

``persist()``

Used for making a new entity managed.

```
java
Employee e = new Employee();

entityManager.persist(e);
```

Conceptually:

```
text
Transient
  ↓
persist
  ↓
Managed
```

``merge()``

Used to copy the state of a detached entity into a managed instance.

Important subtlety:

```
java
Employee managed =
    entityManager.merge(detachedEmployee);
```

The returned object is the managed instance.

The original object may remain detached.

So do **not** assume:

```
java
detached == managed
```

Interview trap

People often say:

merge() makes the original object managed.

That is not the correct mental model.

A better answer:

merge() copies the state of the given entity into a managed entity and returns the managed instance.

Q19. `persist()` vs `save()` in Spring Data JPA?

This distinction is frequently tested.

```
text
Spring Data JPA
    ↓
repository.save()
    ↓
EntityManager
    ↓
persist() / merge()
```

`save()` is a Spring Data repository abstraction.

`persist()` is a JPA API operation.

So they belong to different layers.

Q20. What is `findById()` internally?

Short interview answer

For a normal `JpaRepository`, `findById()` ultimately uses JPA entity lookup through the `EntityManager`, commonly based on `find()`.

Conceptually:

```
text
repository.findById(id)
    ↓
SimpleJpaRepository
    ↓
EntityManager.find(...)
    ↓
Persistence Context
    ↓
Database if needed
```

If the entity is already managed:

```
text
Persistence Context
    ↓
return existing entity
```

Otherwise Hibernate may query the database.

Q21. `findById()` vs `getReferenceById()`?

Very high-value question.

`findById()`

Typically retrieves the actual entity.

```
java
Optional<Employee> employee =
    repository.findById(10L);
```

Hibernate may execute a **SELECT** immediately if the entity isn't already managed.

`getReferenceById()`

Returns a reference/proxy when possible.

```
java
Employee employee =
    repository.getReferenceById(10L);
```

The actual database row may not be fetched immediately.

Conceptually:

```
text
getReferenceById()
    ↓
proxy/reference
    ↓
database access when state is required
```

Example

Suppose:

```

java
Order order = new Order();

Customer customer =
    customerRepository.getReferenceById(5L);

order.setCustomer(customer);

orderRepository.save(order);

```

You may not need to load the complete customer row simply to establish the foreign-key relationship.

Potentially:

```

text
Customer proxy
    ↓
Order.customer_id = 5

```

This can avoid unnecessary SELECTs.

Tricky interviewer question

Does `getReferenceById()` **always avoid SQL?**

No.

It may defer the query, but accessing unloaded state can cause SQL.

Also, depending on the situation, provider behavior and entity state matter.

Q22. What is lazy loading?

Short interview answer

Lazy loading means an associated entity or collection is not loaded from the database until it is actually accessed.

Example:

```

java
@Entity
class Department {

```

```
@OneToMany(fetch = FetchType.LAZY)
private List<Employee> employees;
}
```

When:

```
java
Department d = repository.findById(1L).get();
```

Hibernate may load:

```
text
department
```

but not:

```
text
employees
```

until:

```
java
d.getEmployees().size();
```

Then Hibernate may execute another SQL query.

Conceptually:

```
text
load Department
    ↓
employees not loaded
    ↓
getEmployees()
    ↓
SQL
    ↓
employees loaded
```

Q23. What is eager loading?

Short interview answer

Eager loading means an association is expected to be loaded together with the owning entity, although the exact SQL strategy can depend on the ORM provider and query plan.

Example:

```
java
@ManyToOne(fetch = FetchType.EAGER)
private Department department;
```

The important interview point is:

EAGER does not necessarily mean one SQL query using a JOIN.

Hibernate can use different strategies.

Therefore avoid saying:

EAGER always uses JOIN.

That is an interview trap.

Q24. LAZY vs EAGER — which is better?

Generally, **LAZY is preferred for many associations**, especially collections, because eager loading can unnecessarily retrieve data.

But the correct interview answer is:

Neither is universally better. The choice depends on access patterns. LAZY helps avoid unnecessary loading, while specific query-level fetching such as JOIN FETCH or entity graphs can be used when the related data is required.

A production-friendly pattern is often:

```
text
Default association → LAZY
                        ↓
Query requires relation
                        ↓
JOIN FETCH / EntityGraph
```

instead of globally setting everything to EAGER.

Q25. What is the N+1 query problem?

This is one of the **highest-priority JPA interview questions**.

Suppose we load 100 departments:

```
java
List<Department> departments =
    departmentRepository.findAll();
```

One SQL query:

```
sql
SELECT * FROM department;
```

Then code accesses:

```
java
for (Department d : departments) {
    d.getEmployees().size();
}
```

If employees are lazily loaded, Hibernate may execute:

```
text
1 query → fetch 100 departments

+ 100 queries → fetch employees for each department

= 101 queries
```

Hence:

```
text
N + 1
```

where:

```
text
1 = parent query
N = additional child queries
```

Q26. How do you solve the N+1 problem?

Common solutions:

```
text
JOIN FETCH
```



```
EntityGraph
batch fetching
subselect fetching
DTO projections
careful query design
```

The most common answer:

``JOIN FETCH``

```
java
@Query("""
    SELECT d
    FROM Department d
    JOIN FETCH d.employees
""")
List<Department> findDepartmentsWithEmployees();
```

Conceptually:

```
text
Department
    +
Employees
    ↓
single optimized retrieval strategy
```

But be careful with collection fetch joins and pagination because they can produce duplicate parent rows or problematic pagination behavior.

Another option: EntityGraph

```
java
@EntityGraph(attributePaths = "employees")
List<Department> findAll();
```

This tells JPA/Spring Data what associations should be fetched for that query.

Q27. What is ``JOIN FETCH``?

Short interview answer

`JOIN FETCH` is a JPQL feature that tells the JPA provider to fetch an associated entity as part of the query rather than leaving it for later lazy initialization.

Example:

```
java
@Query("""
    SELECT o
    FROM Order o
    JOIN FETCH o.customer
""")
List<Order> findOrdersWithCustomer();
```

Without fetch join:

```
text
Order query
    ↓
later customer access
    ↓
additional SQL
```

With fetch join:

```
text
Order + Customer
    ↓
retrieved as part of query
```

Critical distinction

```
sql
JOIN
```

and:

```
jpql
JOIN FETCH
```

are not conceptually identical.

`JOIN` participates in query logic.

`JOIN FETCH` additionally instructs the provider to initialize the association as part of the retrieval.

Q28. Why can `JOIN FETCH` cause duplicate results?

Suppose:

text

```
Department A
├─ Employee 1
├─ Employee 2
└─ Employee 3
```

A SQL join can produce:

text

```
Department A | Employee 1
Department A | Employee 2
Department A | Employee 3
```

So the relational result contains repeated department rows.

JPA can reconstruct the entity relationship, but the resulting query semantics may expose duplicates in the Java result depending on the query.

Often developers use:

java

```
SELECT DISTINCT d
FROM Department d
JOIN FETCH d.employees
```

when appropriate.

Tricky interviewer question

Does `DISTINCT` necessarily mean SQL removes all duplicated underlying rows?

Not exactly in the simplistic sense people often assume. JPA/Hibernate query processing can have entity-level de-duplication semantics in addition to SQL `DISTINCT` behavior.

Q29. What is the Persistence Context identity guarantee?

Suppose:

java

```
Employee e1 =
    entityManager.find(Employee.class, 1L);
```

```
Employee e2 =  
    entityManager.find(Employee.class, 1L);
```

Inside the same persistence context, JPA aims to maintain:

```
java  
e1 == e2
```

Why?

Because one persistence context should not represent the same database identity using multiple managed object instances.

Think:

```
text  
  
Persistence Context  
-----  
  
Employee#1 → Object A  
Employee#2 → Object B  
Employee#3 → Object C
```

Another lookup for `Employee#1` resolves to:

```
text  
Object A
```

rather than creating another managed `Employee#1`.

Q30. Why doesn't Hibernate execute an UPDATE immediately when I call a setter?

This question connects several earlier topics.

Suppose:

```
java  
  
Employee employee =  
    entityManager.find(Employee.class, 10L);  
  
employee.setSalary(90000);
```

At this point:

text

Java object changed

not necessarily:

text

SQL UPDATE immediately executed

Hibernate generally uses write-behind behavior through the persistence context.

Conceptually:

text

```
find()
  ↓
managed entity
  ↓
setSalary()
  ↓
memory changed
  ↓
flush
  ↓
dirty checking
  ↓
UPDATE SQL
```

This is why understanding:

text

```
Persistence Context
+
Dirty Checking
+
Flush
```

is much more important than memorizing isolated annotations.

Interview checkpoint — Module 3 Q1–30

At the 2-year level, you should now be able to explain this entire chain without memorized definitions:

text

repository.save(entity)

↓

SimpleJpaRepository

↓

EntityManager

↓

persist / merge

↓

Persistence Context

↓

managed entity

↓

modify entity

↓

Dirty Checking

↓

flush

↓

SQL

↓

transaction commit

And for reads:

text

findById()

↓

EntityManager.find()

↓

Persistence Context?

├ YES → existing managed entity

|

└ NO → Hibernate SQL

And for associations:

text

LAZY

↓

```
not initialized initially
↓
access association
↓
SQL
```

versus:

```
text
JOIN FETCH / EntityGraph
↓
fetch required relationship as part of retrieval
```

And the classic performance problem:

```
text
1 parent query
+
N child queries
=
N+1
```

with typical solutions:

```
text
JOIN FETCH
EntityGraph
batch fetching
DTO projection
query redesign
```

This is the level of understanding I'll maintain for the remaining **50 questions of Module 3**, including **cascading**, `orphanRemoval`, **mappings**, **owning side/inverse side**, `mappedBy`, `@JoinColumn`, **composite keys**, **JPQL vs native SQL**, **pagination**, **projections**, **Specifications**, **transaction propagation**, **isolation**, **optimistic/pessimistic locking**, `@Version`, **batching**, **Hibernate proxies**, **lazy initialization exceptions**, **second-level cache**, **query cache**, **Hibernate SQL generation**, and **real interview scenarios**.

you asked: ok continu

message time: 2026-08-23 03:53:11

ok continue as aboved module 3 including **cascading**, `orphanRemoval`, **mappings**, **owning side/inverse side**, `mappedBy`, `@JoinColumn`, **composite keys**, **JPQL vs native SQL**, **pagination**,

projections, Specifications, transaction propagation, isolation, optimistic/pessimistic locking, @Version, **batching, Hibernate proxies, lazy initialization exceptions, second-level cache, query cache, Hibernate SQL generation, and real interview scenarios.**

chatgpt response: Continuing

Continuing Module 3 from **Q31–80**. I'll keep the same interview depth, but now focus more heavily on mapping internals, performance, transactions/locking, and the traps interviewers use in follow-ups.

Module 3 — JPA / Hibernate

Q31–Q80

We continue directly from **Q1–Q30**.

The next 50 questions move from the basic JPA/Hibernate model into the areas that usually separate a candidate who has only used Spring Data JPA from someone who actually understands how Hibernate behaves in production.

Q31. What is cascading in JPA?

Short interview answer

Cascade means an operation performed on a parent entity is automatically propagated to its associated child entities.

For example:

```
java
@OneToMany(
    mappedBy = "department",
    cascade = CascadeType.ALL
)
private List<Employee> employees;
```

If:

```
java
entityManager.persist(department);
```

then the `persist()` operation can cascade to its employees.

Cascade types

JPA provides:


```
java
```

```
PERSIST  
MERGE  
REMOVE  
REFRESH  
DETACH  
ALL
```

Conceptually:

```
text
```

```
Department  
  |  
  +---- Employee  
  |  
  +---- Employee
```

With:

```
java
```

```
cascade = CascadeType.PERSIST
```

we have:

```
text
```

```
persist (Department)  
      ↓  
persist (Employee)
```

Real project example

Consider:

```
text
```

```
Order  
  ├── OrderItem  
  ├── OrderItem  
  └── OrderItem
```

Usually `OrderItem` belongs exclusively to an `Order`.

You may want:

```
java
@OneToMany(
    mappedBy = "order",
    cascade = CascadeType.ALL
)
private List<OrderItem> items;
```

Then saving the order can save its items.

Tricky interviewer question

Should I always use `CascadeType.ALL`?

No.

Cascade is a domain-modeling decision.

For example, you generally would not want:

```
text
Employee
  ↓
Department
```

to mean that deleting an employee deletes the department.

The cascade direction should match the ownership/lifecycle relationship.

Q32. Explain each cascade type.

‘PERSIST’

```
java
cascade = CascadeType.PERSIST
```

Propagates:

```
java
persist(parent)
```

to child entities.

‘MERGE’

```
java
cascade = CascadeType.MERGE
```

Propagates merge.

``REMOVE``

Propagates delete.

```
java
remove(parent)
    ↓
remove(children)
```

`REMOVE` is intended for `OneToOne` and `OneToMany` associations in portable JPA applications. ([Jakarta EE](#))

``REFRESH``

Refreshes associated entities from the database.

``DETACH``

Detaches associated entities when the parent is detached.

``ALL``

Equivalent to:

```
java
PERSIST
MERGE
REMOVE
REFRESH
DETACH
```

Q33. What is ``orphanRemoval``?

Short interview answer

`orphanRemoval = true` means that when a privately-owned child is removed from its parent's relationship, JPA treats that child as an orphan and schedules it for deletion.

Example:

```
java
@OneToMany(
    mappedBy = "order",
    cascade = CascadeType.ALL,
```

```

        orphanRemoval = true
    )
    private List<OrderItem> items;

```

Suppose:

```

java
order.getItems().remove(item);

```

Then during flush, the removed item can be deleted from the database.

The Jakarta Persistence specification defines orphan removal for `OneToOne` and `OneToMany` associations and says the removal is applied at flush. ([Jakarta EE](#))

Q34. `CascadeType.REMOVE` vs `orphanRemoval`?

This is a **very common interview trap**.

`CascadeType.REMOVE`

The parent is removed:

```

java
entityManager.remove(order);

```

then:

```

text
Order DELETE
    ↓
OrderItem DELETE

```

`orphanRemoval`

The child is removed from the parent's relationship:

```

java
order.getItems().remove(item);

```

then the child itself can be deleted.

So:

```

text
Cascade REMOVE
    parent deleted

```

```

    ↓
    child deleted

orphanRemoval
    child removed from relationship
    ↓
    child deleted

```

Best interview answer

CascadeType.REMOVE propagates an explicit remove operation from parent to child, while orphanRemoval is specifically about deleting a child that has been removed from a privately-owned relationship.

Q35. Does `orphanRemoval = true` mean `cascade = REMOVE` is required?

No.

For an orphaned child, the orphan-removal semantics themselves cause the child removal operation to be applied.

The specification explicitly notes that when removing a managed source entity, cascading remove to a relationship target is handled through orphan-removal semantics, so specifying `cascade=REMOVE` is not necessary for that particular operation. ([Jakarta EE](#))

Still, developers often use:

```

java

cascade = CascadeType.ALL,
orphanRemoval = true

```

because they want both:

```

text

parent lifecycle propagation
+
private child lifecycle

```

Q36. What is the difference between `OneToOne`, `OneToMany`, `ManyToOne`, and `ManyToMany`?

Suppose:

text

```
Department 1 ---- N Employee
Employee   N ---- 1 Department
Student    N ---- N Course
User       1 ---- 1 Profile
```

One-to-one

java

```
@OneToOne
private Profile profile;
```

One entity maps to one entity.

One-to-many

java

```
@OneToMany
private List<Employee> employees;
```

One parent has many children.

Many-to-one

java

```
@ManyToOne
private Department department;
```

Many employees belong to one department.

Many-to-many

java

```
@ManyToMany
private Set<Course> courses;
```

A student can have many courses and a course can have many students.

Q37. Which side is the owning side of a relationship?

Short interview answer

The owning side is the side responsible for maintaining the relationship mapping in the database. In a bidirectional relationship, it is the side that does not use `mappedBy`.

Example:

```

java

@Entity
class Employee {

    @ManyToOne
    @JoinColumn(name = "department_id")
    private Department department;
}

```

and:

```

java

@Entity
class Department {

    @OneToMany(mappedBy = "department")
    private List<Employee> employees;
}

```

Here:

```

text

Employee = owning side
Department = inverse side

```

The JPA specification states that in a bidirectional relationship the inverse side refers to the owning side through `mappedBy`; for bidirectional `OneToMany/ManyToOne`, the many side must be the owning side. ([Jakarta EE](#))

Q38. Why is `ManyToOne` usually the owning side of `OneToMany`?

Because the foreign key normally exists in the many-side table.

Database:

```

text

employee
-----

id
name
department_id ← FK

```

Therefore:

```

text
Employee
  ↓
department_id

```

is the side that knows which department it belongs to.

Hence:

```

java
@ManyToOne
@JoinColumn(name = "department_id")
private Department department;

```

while the other side uses:

```

java
@OneToMany(mappedBy = "department")
private List<Employee> employees;

```

Database reasoning

```

text
Department
  1
  |
  |
  N
Employee

```

The employee row stores:

```

text
department_id

```

so the employee association controls the FK.

Q39. What is `mappedBy`?

Short interview answer

`mappedBy` tells JPA that the current side is the inverse side and that the actual relationship is owned by the specified field on the other entity.

Example:


```

java
@Entity
class Department {

    @OneToMany(mappedBy = "department")
    private List<Employee> employees;
}

```

Here:

```

text
"department"

```

refers to:

```

java
Employee.department

```

So:

```

text
Department.employees
    ↓
mappedBy = "department"
    ↓
Employee.department

```

The `mappedBy` value refers to the **Java entity field/property**, not the database column name. ([Jakarta EE](#))

Q40. What happens if I don't use `mappedBy` on a bidirectional relationship?

Hibernate/JPA may interpret both sides as separate relationship mappings.

For example:

```

java
Department:
@OneToMany
List<Employee> employees;

Employee:

```

```
@ManyToOne
Department department;
```

Without correctly declaring ownership, you can end up with:

```
text
additional join table
duplicate mappings
unexpected SQL
schema problems
relationship not behaving as expected
```

The correct mapping is usually:

```
java
Department:
@OneToMany(mappedBy = "department")

Employee:
@ManyToOne
@JoinColumn(name = "department_id")
```

Q41. What is `@JoinColumn`?

Short interview answer

`@JoinColumn` specifies the foreign-key column used to map an entity association.

Example:

```
java
@ManyToOne
@JoinColumn(name = "department_id")
private Department department;
```

Database:

```
text
employee
-----
id
name
department_id
```

JPA knows:

```
text
Employee.department
    ↓
employee.department_id
    ↓
Department.id
```

`@JoinColumn` is defined by Jakarta Persistence as specifying the column used to join an entity association or element collection. ([Jakarta EE](#))

Q42. What is `referencedColumnName` in `@JoinColumn`?

Example:

```
java
@JoinColumn(
    name = "department_code",
    referencedColumnName = "code"
)
```

This means:

```
text
employee.department_code
    ↓
department.code
```

rather than assuming the target's primary-key column.

Default behavior typically references the target entity's primary key when no `referencedColumnName` is specified.

Tricky question

Should I use `referencedColumnName` everywhere?

No.

For the normal primary-key relationship:

```
java
@JoinColumn(name = "department_id")
```

is often enough.

Q43. What is an unidirectional vs bidirectional relationship?

Unidirectional

Only one entity knows about the relationship.

```
java
class Employee {

    @ManyToOne
    private Department department;

}
```

There is no:

```
java
Department.employees
```

Bidirectional

Both entities have references:

```
java
Employee.department
```

and:

```
java
Department.employees
```

Important:

Bidirectional in Java does not automatically mean both sides update themselves.

You must keep both sides synchronized in application code.

For example:

```
java
department.getEmployees().add(employee);
employee.setDepartment(department);
```

The specification explicitly notes that the application is responsible for maintaining consistency of both sides of a bidirectional relationship in runtime state. ([Jakarta EE](#))

Q44. Why should we maintain both sides of a bidirectional relationship?

Suppose:

```
java
department.getEmployees().add(employee);
```

but you forget:

```
java
employee.setDepartment(department);
```

Your Java object graph says:

```
text
Department → Employee
```

but:

```
text
Employee → Department
```

may still be `null`.

This can create bugs even if the database eventually stores the correct relationship.

A common helper method is:

```
java
public void addEmployee(Employee employee) {
    employees.add(employee);
    employee.setDepartment(this);
}
```

and:

```
java
public void removeEmployee(Employee employee) {
    employees.remove(employee);
    employee.setDepartment(null);
}
```

This becomes especially important with `orphanRemoval`.

Q45. Explain composite primary keys in JPA.

A composite key means one row is identified by multiple columns.

Example:

```
text
employee_project
-----
employee_id
project_id
```

Together:

```
text
(employee_id, project_id)
```

form the primary key.

JPA supports approaches such as:

```
text
@EmbeddedId
@IdClass
```

Q46. What is `@EmbeddedId`?

Example:

```
java
@Embeddable
public class EmployeeProjectId {

    private Long employeeId;
    private Long projectId;

}
```

Then:

```
java
@Entity
public class EmployeeProject {

    @EmbeddedId
    private EmployeeProjectId id;

}
```

Database:

```
text
employee_project
-----
employee_id
project_id
```

The embedded ID represents the composite key as a value object.

Why this is useful

It gives you:

```
java
EmployeeProjectId id =
    new EmployeeProjectId(employeeId, projectId);
```

and the identity is represented by one Java object.

Q47. What is `@IdClass`?

Example:

```
java
@Entity
@IdClass(EmployeeProjectId.class)
public class EmployeeProject {

    @Id
    private Long employeeId;

    @Id
    private Long projectId;

}
```

The ID class contains the key fields as well.

Conceptually:

```
text
@IdClass
```

```
↓  
multiple @Id fields directly on entity
```

while:

```
text  
@EmbeddedId  
↓  
one @EmbeddedId object
```

Interview comparison

@EmbeddedId	@IdClass
Composite key wrapped in object	Composite ID represented by multiple entity fields
Key is embedded	Key fields directly on entity
Often cleaner for value-object modeling	Can be convenient when fields are frequently accessed directly

Q48. What must a composite key class provide?

The key class must properly represent identity and therefore requires appropriate:

```
text  
equals()  
hashCode()
```

and appropriate constructor/access characteristics according to the JPA mapping approach.

Why?

Because the persistence provider and Java collections need a stable notion of entity identity.

For an interview:

Composite keys require consistent equality and hashing because the combined key determines entity identity.

Q49. JPQL vs native SQL?

JPQL

JPQL operates on:


```
text
entities
+
entity attributes
```

Example:

```
java
@Query("""
    SELECT e
    FROM Employee e
    WHERE e.salary > :salary
""")
List<Employee> findHighEarners(double salary);
```

Notice:

```
text
Employee
e.salary
```

not table/column names.

Native SQL

```
java
@Query(
    value = ""
    SELECT *
    FROM employee
    WHERE salary > :salary
    "",
    nativeQuery = true
)
List<Employee> findHighEarners(double salary);
```

Here you write database SQL.

Comparison

```
text
JPQL
↓
```

```
portable entity-oriented query
```

```
Native SQL
```

```
↓
```

```
database-oriented query
```

Q50. When should you use native SQL instead of JPQL?

Use native SQL when:

text

```
complex database-specific features  
vendor-specific functions  
CTEs/window functions where appropriate  
database hints  
specialized reporting queries  
performance-sensitive SQL requiring exact control  
legacy SQL/database structures
```

But don't use native SQL simply because JPQL feels unfamiliar.

Interview answer

I prefer JPQL for normal entity-oriented operations because it is portable and works with the ORM model. I use native SQL when the database-specific capability or query shape isn't practical or expressible cleanly in JPQL.

Q51. What is Criteria API?

Short interview answer

Criteria API is a programmatic, type-oriented way of building JPA queries dynamically rather than concatenating query strings.

Example conceptually:

java

```
CriteriaBuilder cb =  
    entityManager.getCriteriaBuilder();  
  
CriteriaQuery<Employee> query =  
    cb.createQuery(Employee.class);
```

You can construct predicates dynamically:

```
text
if salary exists → add salary condition
if department exists → add department condition
if name exists → add name condition
```

This is useful for dynamic search/filter functionality.

Disadvantage

Criteria code can become verbose.

Q52. What are Spring Data JPA Specifications?

Short interview answer

Specifications provide a reusable way to construct dynamic database predicates using Spring Data JPA's Criteria API integration.

Example:

```
java
public static Specification<Employee> salaryGreaterThan(
    BigDecimal salary) {

    return (root, query, cb) ->
        cb.greaterThan(root.get("salary"), salary);
}
```

Then:

```
java
employeeRepository.findAll(
    salaryGreaterThan(BigDecimal.valueOf(100000))
);
```

Repository:

```
java
public interface EmployeeRepository
    extends JpaRepository<Employee, Long>,
        JpaSpecificationExecutor<Employee> {
}
```

Q53. When are Specifications useful?

Imagine a search screen:

```
text
name          optional
department    optional
minSalary     optional
maxSalary     optional
joiningDate   optional
status        optional
```

Without Specifications, you might need many repository methods:

```
text
findByName
findByNameAndDepartment
findByNameAndSalary
findByDepartmentAndSalary
...
```

Specifications allow you to dynamically compose:

```
text
Specification A
    AND
Specification B
    AND
Specification C
```

This is a strong real-project use case.

Q54. What is pagination in Spring Data JPA?

Instead of retrieving:

```
text
1,000,000 rows
```

you retrieve a page:

```
text
page 0 → 20 rows
```

```
page 1 → 20 rows
page 2 → 20 rows
```

Example:

```
java
Pageable pageable =
    PageRequest.of(0, 20);

Page<Employee> page =
    employeeRepository.findAll(pageable);
```

You get information such as:

```
java
page.getContent();
page.getTotalElements();
page.getTotalPages();
page.getNumber();
```

Q55. How does pagination work internally?

Conceptually:

```
text
PageRequest
    ↓
Spring Data JPA
    ↓
Hibernate query
    ↓
SQL LIMIT/OFFSET
```

The actual SQL syntax depends on the database.

For example:

```
sql
SELECT ...
FROM employee
ORDER BY id
LIMIT 20 OFFSET 40;
```

Spring Data commonly also executes a count query for total-element information when returning a Page.

Performance concern

For very large offsets:

```
text
OFFSET 10,000,000
```

can become expensive.

That leads to:

```
text
keyset / cursor pagination
```

Q56. What is keyset pagination?

Instead of:

```
sql
OFFSET 1,000,000
```

use the last seen key.

Example:

```
sql
SELECT *
FROM employee
WHERE id > :lastId
ORDER BY id
LIMIT 20;
```

Then:

```
text
page 1 → IDs 1-20
page 2 → WHERE id > 20
page 3 → WHERE id > 40
```

This is often much more scalable for sequential navigation over large datasets.

Interview answer

Offset pagination is simple and convenient but can become costly at large offsets. Keyset pagination uses a stable ordering key and starts from the last-seen value, making it more suitable for large datasets.

Q57. What are projections in Spring Data JPA?

Projections allow you to retrieve only the required data rather than the entire entity.

Suppose:

```
text
Employee
-----
id
name
salary
address
department
...
```

But API needs only:

```
text
id
name
```

Instead of loading the complete entity, use a projection.

Example:

```
java
public interface EmployeeSummary {

    Long getId();

    String getName();

}
```

Repository:

```
java
List<EmployeeSummary>
findByDepartment(String department);
```

Benefit

Potentially:

```
text
less data
less object creation
less memory usage
better query efficiency
```

Q58. What are DTO projections?

Instead of returning an entity:

```
java
SELECT e
FROM Employee e
```

you can select a DTO:

```
java
SELECT new com.example.EmployeeDto(
    e.id,
    e.name,
    e.salary
)
FROM Employee e
```

This can be particularly useful for:

```
text
read APIs
reporting
large queries
aggregation
service-layer boundaries
```

Important interview point

A DTO projection is not the same as an entity.

It is generally not managed by the persistence context like a normal entity.

Q59. Why should we avoid returning JPA entities directly from REST APIs?

This is a very practical product-company interview question.

Suppose:

```
java
@GetMapping("/employees")
public List<Employee> getEmployees() {
    return employeeRepository.findAll();
}
```

Potential problems:

```
text
entity structure exposed
lazy-loading issues
recursive relationships
unwanted columns
API tightly coupled to DB model
serialization of associations
N+1 during serialization
security/data exposure
```

Better:

```
text
Entity
  ↓
Mapper
  ↓
DTO
  ↓
REST response
```

This creates a cleaner API boundary.

Q60. What is a Hibernate proxy?

Short interview answer

A Hibernate proxy is a generated object that stands in for an entity or association and can defer loading until the actual entity state is needed.

Example:

```
java
Employee employee =
    entityManager.getReference(Employee.class, 10L);
```

Hibernate may return a proxy conceptually like:

```
text
Employee proxy
    |
    +-- id = 10
    |
    +-- other fields not loaded yet
```

When you access required state:

```
java
employee.getName();
```

Hibernate may initialize the proxy and execute SQL.

Important point

Hibernate's implementation details can differ by version and bytecode enhancement setup, so don't make an interview claim that every lazy association is always represented by exactly the same proxy mechanism.

Q61. What is `LazyInitializationException`?

Short interview answer

`LazyInitializationException` occurs when Hibernate needs to initialize a lazy association, but the persistence context/session is no longer available.

Classic example:

```
java
@Transactional
public Order getOrder(Long id) {
    return repository.findById(id).get();
}
```

Transaction ends.

Later:

```
java
order.getItems().size();
```

If `items` was LAZY and wasn't initialized:

```
text
Session closed
    ↓
Hibernate tries to initialize collection
    ↓
can't access persistence context
    ↓
LazyInitializationException
```

Q62. How do you solve `LazyInitializationException`?

Don't blindly make everything:

```
java
FetchType.EAGER
```

Better solutions include:

Solution 1 — fetch required data inside transaction

```
java
@Transactional
public OrderDto getOrder(Long id) {

    Order order =
        repository.findOrderWithItems(id);

    return mapper.toDto(order);
}
```

Solution 2 — `JOIN FETCH`

```
java
@Query("""
```

```

SELECT o
FROM Order o
JOIN FETCH o.items
WHERE o.id = :id
""")
Optional<Order> findOrderWithItems(Long id);

```

Solution 3 — EntityGraph

```

java
@EntityGraph(attributePaths = "items")
Optional<Order> findById(Long id);

```

Solution 4 — DTO projection

Fetch exactly what the API needs.

Bad simplistic solution

```

text
Make everything EAGER

```

This can create massive performance problems.

Q63. What is the Open Session in View pattern?

Short interview answer

Open Session in View keeps the persistence context available during web request processing so lazy relationships can be initialized later, including during view/response processing.

It can make `LazyInitializationException` less visible.

But it can also hide poor query design.

For REST applications, blindly relying on it can result in:

```

text
controller serialization
    ↓
lazy association access
    ↓
unexpected SQL
    ↓
N+1

```

A stronger approach is to design explicit fetch plans at the service/repository layer.

Q64. What is the Second-Level Cache?

Short interview answer

The second-level cache is a Hibernate cache associated with the persistence unit/session factory and shared across persistence contexts.

Contrast:

```
text
First-level cache
    ↓
per Persistence Context

Second-level cache
    ↓
shared across Persistence Contexts
```

Conceptually:

```
text
Transaction A
    ↓
Session A
    ↓
    Second-Level Cache
    ↑
Session B
    ↑
Transaction B
```

Hibernate supports second-level caching through configured cache regions/provider infrastructure.

Important

Second-level cache is **not automatically appropriate for every entity**.

It works best for data with:

```
text
high read frequency
```

```
low mutation frequency
predictable consistency requirements
```

Q65. First-Level Cache vs Second-Level Cache?

First-level	Second-level
Persistence Context/Session scoped	SessionFactory/persistence-unit scoped
Built into normal Hibernate session behavior	Requires configuration/cache provider
Always associated with current persistence context	Shared across persistence contexts
Entity identity guarantee	Reuse of cached state across sessions

The mental model:

```
text
L1:
Current transaction/session

L2:
Application-wide Hibernate persistence infrastructure
```

Q66. What is Hibernate Query Cache?

Short interview answer

Query cache stores query result information so that repeated execution of certain queries can avoid repeating the same database lookup, typically working together with the second-level cache.

Important distinction:

```
text
Entity cache
≠
Query result cache
```

For example:

```
text
Query:
find employees where department = IT
```

The cache may remember the identifiers returned by the query.

The entity state itself may then come from the second-level cache.

Interview trap

Query cache is not a magical cache for all SQL.

It has invalidation and consistency considerations and can add complexity.

Use it selectively.

Q67. What is Hibernate batching?

There are two important meanings candidates often mix up:

```
text
JDBC statement batching
```

and:

```
text
batch fetching
```

JDBC batching

Hibernate groups multiple INSERT/UPDATE statements into JDBC batches.

Conceptually:

```
text
INSERT
INSERT
INSERT
INSERT
```

becomes a batched operation sent efficiently to the driver/database.

Hibernate exposes configuration such as:

```
properties
hibernate.jdbc.batch_size=25
```

Hibernate's documentation explains that JDBC batching can reduce database/network round trips by grouping statements. ([Hibernate Documentation](#))

Q68. What is batch fetching?

Batch fetching is different.

Suppose:

```
text
Department 1
Department 2
Department 3
Department 4
```

and each has a lazy relationship.

Instead of:

```
text
load department 1
load employees 1

load department 2
load employees 2

load department 3
load employees 3
```

Hibernate may batch multiple identifiers into one query.

Hibernate supports mechanisms such as:

```
java
@BatchSize(size = 10)
```

for batch fetching. ([Hibernate Documentation](#))

So remember:

```
text
JDBC batching
    ↓
batch DML statements

Batch fetching
    ↓
batch SELECT loading
```


Q69. What is `hibernate.jdbc.batch\_size`?

It controls the maximum number of JDBC statements Hibernate can group into a batch before sending execution to the JDBC driver.

For example:

```
properties
hibernate.jdbc.batch_size=25
```

means Hibernate can batch up to roughly 25 statements in a batch, subject to the actual operation/query shape and driver/database behavior. Hibernate documents zero or negative values as disabling JDBC batching. ([Hibernate Documentation](#))

Practical example

Suppose you import 10,000 employees.

Without batching:

```
text
INSERT
network round trip
INSERT
network round trip
...
```

With batching:

```
text
25 INSERT statements
    ↓
one JDBC batch execution
```

This can substantially improve bulk insert performance.

Q70. Why can Hibernate batching fail to give the expected performance?

Because batching is not just:

```
properties
hibernate.jdbc.batch_size=50
```

Performance also depends on:

text

```
database driver
database capabilities
generated ID strategy
flush frequency
transaction size
entity ordering
network latency
indexes
constraints
SQL shape
memory usage
```

A particularly important issue is generated identifiers. Certain ID strategies can affect insert batching.

So an excellent interview answer is:

Enabling JDBC batching is necessary but not sufficient. I would also verify the ID generation strategy, flush/clear frequency, transaction boundaries, SQL produced, and actual database execution plan/metrics.

Q71. How do you insert 100,000 records efficiently with Hibernate?

A naive implementation:

```
java
for (...) {
    entityManager.persist(entity);
}
```

may cause the persistence context to grow very large.

A better batching approach:

```
java
for (int i = 0; i < employees.size(); i++) {

    entityManager.persist(employees.get(i));

    if (i % 50 == 0) {
        entityManager.flush();
        entityManager.clear();
    }
}
```

```
}  
}
```

Conceptually:

```
text  
50 entities  
  ↓  
flush  
  ↓  
clear  
  
50 entities  
  ↓  
flush  
  ↓  
clear
```

Why `clear()`?

`flush()` sends pending changes.

`clear()` removes managed entities from the persistence context, preventing memory from growing unnecessarily.

Interview trap

```
text  
flush()
```

does not mean:

```
text  
clear()
```

They are separate operations.

Q72. What is optimistic locking?

Short interview answer

Optimistic locking assumes conflicts are relatively uncommon and detects concurrent modifications using a version field rather than holding a database lock for the whole business operation.

Example:

```
java
@Version
private Long version;
```

Database:

```
text
id | name | salary | version
-----
10 | A    | 50000  | 3
```

Transaction A reads:

```
text
version = 3
```

Transaction B reads:

```
text
version = 3
```

Transaction A updates:

```
sql
UPDATE employee
SET salary = 60000,
    version = 4
WHERE id = 10
AND version = 3;
```

Transaction B then tries:

```
sql
UPDATE employee
SET salary = 70000,
    version = 4
WHERE id = 10
AND version = 3;
```

No row matches because version is now:

text

4

Therefore Hibernate can detect the optimistic locking conflict.

Jakarta Persistence defines optimistic locking around version/timestamp verification and requires an `OptimisticLockException` when the version check fails. ([Jakarta EE](#))

Q73. What is `@Version`?

Example:

```
java
@Entity
public class Employee {

    @Id
    private Long id;

    @Version
    private Long version;
}
```

It represents the entity revision used for optimistic concurrency control.

Supported version types include:

```
text
int / Integer
short / Short
long / Long
Timestamp
Instant
LocalDateTime
```

according to the current Jakarta Persistence API documentation. ([Jakarta EE](#))

Important

You should not manually do:

```
java
employee.setVersion(employee.getVersion() + 1);
```

The persistence provider manages version updates.

Q74. What exactly does `@Version` prevent?

Primarily:

```
text
lost updates
```

Example:

```
text

Initial:
salary = 50,000
version = 1

Transaction A:
reads 1

Transaction B:
reads 1

A updates → version 2

B tries update with version 1
    ↓
conflict
    ↓
OptimisticLockException
```

Without versioning, B might silently overwrite A's work.

Q75. What is pessimistic locking?

Short interview answer

Pessimistic locking obtains a database-level lock when the entity is read/locked so other transactions are prevented from modifying it in conflicting ways until the lock-holding transaction ends.

Examples:

```
java
LockModeType.PESSIMISTIC_READ
LockModeType.PESSIMISTIC_WRITE
LockModeType.PESSIMISTIC_FORCE_INCREMENT
```

For example:

```
java
Employee employee =
    entityManager.find(
        Employee.class,
        id,
        LockModeType.PESSIMISTIC_WRITE
    );
```

The exact database mechanism is provider/database dependent. Jakarta Persistence defines the semantics but does not require one universal database locking implementation. ([Jakarta EE](#))

Q76. Optimistic vs pessimistic locking — when would you use each?

Optimistic	Pessimistic
Low/moderate contention	High contention
No long-lived DB lock	Explicit DB locking
Better concurrency	More blocking
Uses version/checking	Uses DB locks
Conflict detected later	Conflict prevented/blocked earlier

Example: optimistic

Normal employee profile update:

```
text
many reads
few conflicting updates
```

Use:

```
java
@Version
```

Example: pessimistic

Inventory allocation:

```
text
stock = 1
```

Two transactions trying to reserve the final item may require stronger coordination.

Potentially:

```
text
SELECT ... FOR UPDATE
```

through the provider/database locking mechanism.

Interview answer

I prefer optimistic locking for typical business entities with moderate contention because it scales better and avoids long-held database locks. I would consider pessimistic locking when contention is high and conflicting transactions must be serialized.

Q77. What is transaction isolation?

Transaction isolation defines how one transaction can observe changes made by other concurrent transactions.

Common levels:

```
text
READ_UNCOMMITTED
READ_COMMITTED
REPEATABLE_READ
SERIALIZABLE
```

Conceptually:

```
text
READ_UNCOMMITTED
  ↓
maximum concurrency / weakest isolation

READ_COMMITTED
  ↓
REPEATABLE_READ
  ↓
```



```
SERIALIZABLE
```

```
↓
```

```
strongest isolation / potentially lowest concurrency
```

Typical anomalies:

```
text
```

```
Dirty read
```

```
Non-repeatable read
```

```
Phantom read
```

Q78. Dirty read vs non-repeatable read vs phantom read?

Dirty read

Transaction A reads data written by B before B commits.

```
text
```

```
B → UPDATE
```

```
A → reads uncommitted value
```

```
B → ROLLBACK
```

A saw data that never became committed.

Non-repeatable read

Transaction A reads the same row twice and gets different committed values because another transaction changed it between reads.

```
text
```

```
A → read salary = 50k
```

```
B → update salary = 60k + commit
```

```
A → read salary = 60k
```

Phantom read

Transaction A repeats a query and sees additional/removing rows because another transaction inserted/deleted matching rows.

```
text
```

```
A → SELECT employees WHERE salary > 50k
```

```
B → INSERT salary = 70k + commit
```

```
A → same SELECT
```

```
↓  
extra row appears
```

Q79. What is transaction propagation in Spring?

This connects JPA/Hibernate with Spring transactions.

The most common propagation modes are:

```
text  
  
REQUIRED  
REQUIRES_NEW  
SUPPORTS  
MANDATORY  
NOT_SUPPORTED  
NEVER  
NESTED
```

`REQUIRED`

```
java  
  
@Transactional(propagation = Propagation.REQUIRED)
```

If transaction exists:

```
text  
  
join existing
```

Otherwise:

```
text  
  
create transaction
```

This is the default.

`REQUIRES\_NEW`

Suspends existing transaction and creates a completely new transaction.

```
text  
  
T1  
|  
+-- call service  
  
↓
```

```
suspend T1
↓
T2
↓
commit/rollback T2
↓
resume T1
```

Real project example

Suppose:

```
text
Order transaction
    ↓
Audit service
```

You may want audit persistence in a separate transaction under some business requirements:

```
java
@Transactional(propagation = Propagation.REQUIRES_NEW)
public void saveAudit(...) {
}
```

Then audit transaction can commit independently of the outer transaction.

Tricky interviewer question

Does `REQUIRES_NEW` mean nested transaction?

No.

It means:

```
text
suspend current transaction
+
start separate transaction
```

It is not the same concept as a database savepoint/nested transaction.

Q80. Explain this complete JPA/Hibernate production scenario.

This is the kind of scenario where an interviewer may combine 10 different questions into one.

Scenario

You have:

```
text
Order
├─ Customer
└─ OrderItems
    ├─ Product
    ├─ Product
    └─ Product
```

The API:

```
http
GET /orders/100
```

is slow.

You investigate and find:

```
text
1 query → Order

then:
1 query → Customer
3 queries → Products
```

or perhaps many more queries because of lazy associations.

How would you debug it?

First inspect:

```
text
SQL logs
Hibernate statistics
query count
execution time
DB execution plan
```

Then ask:

```
text
Is this N+1?
```

```
Which association triggered it?  
Is LAZY appropriate?  
What data does the API actually require?
```

Suppose the endpoint needs:

```
text  
  
order id  
order date  
customer name  
product names  
quantities
```

Rather than blindly changing:

```
java  
FetchType.LAZY → EAGER
```

I would consider:

```
text  
DTO projection
```

or:

```
java  
JOIN FETCH
```

or:

```
java  
@EntityGraph
```

depending on the query and result shape.

For example:

```
java  
@Query("""  
    SELECT DISTINCT o  
    FROM Order o  
    JOIN FETCH o.customer  
    JOIN FETCH o.items i  
    JOIN FETCH i.product
```

```
WHERE o.id = :id
""")
Optional<Order> findOrderDetails(Long id);
```

Then map to:

```
text
Order
↓
OrderResponse DTO
```

But there is another trap

Fetching multiple collection relationships through one giant join can produce a Cartesian-product-like row explosion.

For example:

```
text
Order
  10 items
  5 tags
```

could produce:

```
text
10 × 5 = 50 joined rows
```

So the interviewer may ask:

"Would you always use JOIN FETCH to solve N+1?"

The correct answer is:

No. JOIN FETCH is one solution, but the query shape matters. Fetching multiple collections can multiply rows and create large result sets. I would choose between fetch joins, entity graphs, batch fetching, DTO projections, or multiple focused queries based on the access pattern.

That is the kind of answer that demonstrates actual Hibernate understanding.

★ High-value interview scenarios you should now know

Scenario 1 — `save()` followed by update

java

```
Employee e = new Employee();  
repository.save(e);  
  
e.setSalary(100000);
```

Question:

Why can the update happen without calling save() again?

Answer:

text

```
save()  
↓  
persist()  
↓  
managed entity  
↓  
setter changes managed object  
↓  
dirty checking  
↓  
flush  
↓  
UPDATE
```

Scenario 2 — `findById()` called twice

java

```
Employee e1 = repository.findById(10L).get();  
Employee e2 = repository.findById(10L).get();
```

Question:

Will Hibernate always execute two SELECTs?

No.

Within the same persistence context, the first-level cache/persistence context can return the already managed entity.

Scenario 3 — `LAZY` causes exception

```

java
Order order = service.getOrder(1L);

return order.getItems();

```

Question:

Why does this sometimes fail?

Because:

```

text
transaction/session ended
    ↓
lazy collection not initialized
    ↓
access association
    ↓
LazyInitializationException

```

Scenario 4 — `EAGER` makes application slower

Question:

"I had an N+1 issue, so I changed everything to EAGER. Now why is the application slower?"

Because EAGER can force associated data to be loaded even when it isn't needed.

Possible result:

```

text
large joins
large object graphs
more memory
more SQL
unwanted data

```

Better:

```

text
LAZY by default
+
explicit fetch plan for each use case

```


Scenario 5 — Two users update the same record

Initial:

```
text
salary = 50,000
version = 4
```

Both users load version 4.

User A updates first:

```
text
version → 5
```

User B attempts update with version 4.

Hibernate detects:

```
text
expected version = 4
database version = 5
```

and raises an optimistic locking failure.

This is exactly the scenario `@Version` is designed to address. ([Jakarta EE](#))

☐ The 20 concepts from Q31–Q80 you must be able to explain without notes

```
text
Cascade
CascadeType.ALL
CascadeType.REMOVE
orphanRemoval
OneToOne
OneToMany
ManyToOne
ManyToMany
owning side
inverse side
mappedBy
@JoinColumn
```

```
unidirectional
bidirectional
composite key
@EmbeddedId
@IdClass
JPQL
Native SQL
Criteria API
Specifications
Pagination
Keyset pagination
Projection
DTO projection
Hibernate proxy
LazyInitializationException
Second-level cache
Query cache
JDBC batching
Batch fetching
Optimistic locking
@Version
Pessimistic locking
Isolation levels
Dirty read
Non-repeatable read
Phantom read
Transaction propagation
REQUIRED
REQUIRES_NEW
N+1
JOIN FETCH
```

The most important mental model for Module 3

A strong interviewer may not ask these concepts separately. They may give you one production problem and expect you to connect all of them:

text

REST API

↓

```
Service
  ↓
@Transactional
  ↓
Repository
  ↓
Spring Data JPA
  ↓
EntityManager
  ↓
Persistence Context
  ↓
Hibernate
  ├── First-Level Cache
  ├── Dirty Checking
  ├── Proxies
  ├── Lazy Loading
  ├── Fetch Planning
  ├── Batching
  ├── Second-Level Cache
  └── SQL Generation
  ↓
JDBC
  ↓
Database
```

And when something goes wrong:

```
text
Slow API
  ↓
Too many SQL queries?
  ↓
N+1?
  ↓
Wrong fetch strategy?
  ↓
JOIN FETCH?
  ↓
```

```

DTO projection?
  ↓
Batch fetching?
  ↓
Pagination?
  ↓
DB execution plan?

```

Or:

```

text
Concurrent update
  ↓
Lost update?
  ↓
@Version?
  ↓
Optimistic locking?
  ↓
High contention?
  ↓
Pessimistic locking?

```

Or:

```

text
Delete parent
  ↓
Should children disappear?
  ↓
Cascade REMOVE?
  ↓
Child removed from collection?
  ↓
orphanRemoval?

```

That **connection between concepts** is what you should practice for a 2-year product-company interview, rather than memorizing annotations individually.